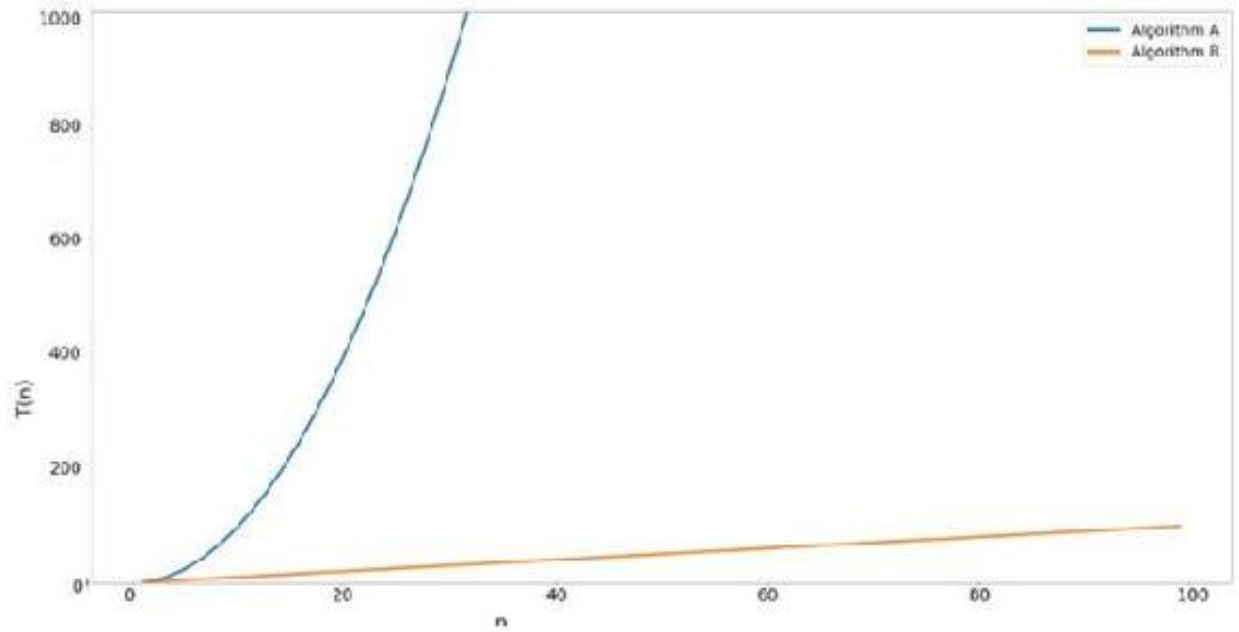


Figure B.1 Visual comparison of the running time of a quadratic algorithm (A), and a linear algorithm (B). The former grows so much faster that, although the y axes shows up to 10 times the max value reached by algorithm B, it only manages to show the plot for algorithm A up to n~=30.

Also, as suggested, small variations in the number of steps executed are hardly relevant. We could rather reason on order of magnitudes changes: 2x, 10x, 100x and so on.

But to really understand when an algorithm performs better than another one on a given problem instance, we need another epiphany: we need to express the number of steps performed as a function of the problem size.

Suppose we are comparing two sorting algorithms and we state that "on a given test set, the first algorithm takes 100 steps to get to the solution while the second one takes 10".